

Grading

 We will grade your assignments based on **test results and design quality**.

Testing results (60%)

In section [Testing](#), you use `make check` to run our test suite on your Pintos implementation, but it only tells you each test case passes or fails. During your development, this is definitely ok, because your goal is to pass tests as many as possible.

But after your development, we need a way to assign different weights to these test cases and collect all the scores as your final testing grade. And this is exactly what `make grade` does. Below, let me walk you through how `make grade` works, taking `Lab1: threads` as an example.

In `src/threads/Make.vars` :

```
kernel.bin: DEFINES =  
KERNEL_SUBDIRS = threads devices lib lib/kernel $(TEST_SUBDIRS)  
TEST_SUBDIRS = tests/threads  
GRADING_FILE = $(SRCDIR)/tests/threads/Grading  
SIMULATOR = --bochs
```

`TEST_SUBDIRS` defines the test directories included and `GRADING_FILE` defines the grading file:

`src/tests/threads/Grading` :

```
20.0%    tests/threads/Rubric.alarm
40.0%    tests/threads/Rubric.priority
40.0%    tests/threads/Rubric.mlfqs
```

`make grade` will compile and run all the test cases included in `TEST_SUBDIRS` and collect the passed scores for testing set `alarm`, `priority` and `mlfqs`. Then it will calculate the final grade based on the percentage defined in the grading file.

We require you to write down your `make grade` score in your design document and we will contact you if your expected score has a huge difference from our testing result for you.

Notes for Windows

In windows, if you issue `make grade` to check your score locally, you may encounter the error: `/bin/sh: 1: ../../tests/make-grade: not found, due to the CRLF problem.`

In order to solve this problem, you can download the tool `dos2unix` [here ↗](#), and follow the instructions in [this blog ↗](#) to get you out of the trouble.

Design Doc & Coding style results (40%)

 **We will judge your design based on the design document and the source code that you submit.** We will read your entire design document and much of your source code.

Don't forget that design quality, including the design document, is 40% of your project grade. It is better to spend one or two hours writing a good design document than spend that time getting the last 5% of the points for tests and then try to rush through writing the design document in the last 15 minutes.

Design Document

We provide a design document template for each project. For each significant part of a project, the template asks questions in four areas:

Data Structures

The instructions for this section are always the same:

Copy here the declaration of each new or changed `struct` or `struct` member, global or static variable, `typedef`, or enumeration. Identify the purpose of each in 25 words or less.

- The first part is mechanical. Just **copy new or modified declarations into the design document, to highlight for us the actual changes to data structures**. Each declaration should include the comment that should accompany it in the source code (see below).
- We also ask for a very brief description of the purpose of each new or changed data structure. **The limit of 25 words or less is a guideline intended to save your time and avoid duplication with later areas.**

Algorithms

This is where you tell us how your code works, through questions that probe your understanding of your code.

- We might not be able to easily figure it out from the code, because many creative solutions exist for most OS problems. Help us out a little.
- **Your answers should be at a level below the high-level description of the requirements given in the assignment.** We have read the assignment too, so it is unnecessary to repeat or rephrase what is stated there.



- **On the other hand, your answers should be at a level above the low level of the code itself.** Don't give a line-by-line run-down of what your code does. Instead, use your answers to explain how your code works to implement the requirements.

Synchronization

An operating system kernel is a complex, multithreaded program, in which synchronizing multiple threads can be difficult. **This section asks about how you synchronize a particular type of activity.**

Rationale

Whereas the other sections primarily ask "what" and "how," the rationale section concentrates on "why." **This is where we ask you to justify some design decisions, by explaining why the choices you made are better than alternatives.**

- You may be able to state these **in terms of time and space complexity**, which can be made as rough or informal arguments (formal language or proofs are unnecessary).
- An incomplete, evasive, or non-responsive design document or one that strays from the template without good reason may be penalized. Incorrect capitalization, punctuation, spelling, or grammar can also cost points. See section [Project Documentation](#), for a sample design document for a fictitious project.

Code Style

Your design will also be judged by looking over your source code.

- We will typically look at **the differences between the original Pintos source tree and your submission**, based on the output of a command like `diff -urpb pintos.orig pintos.submitted`.



- We will try to **match up your description of the design with the code submitted**. Important discrepancies between the description and the actual code will be penalized, as will be any bugs we find by spot checks.

The most important aspects of source code design are those that specifically relate to the operating system issues at stake in the project.

- For example, the organization of an inode is an important part of file system design, so in the file system project a poorly designed inode would lose points.
- Other issues are much less important. For example, multiple Pintos design problems call for a "priority queue," that is, a dynamic collection from which the minimum (or maximum) item can quickly be extracted. Fast priority queues can be implemented many ways, but **we do not expect you to build a fancy data structure** even if it might improve performance. Instead, you are welcome to use a linked list (and Pintos even provides one with convenient functions for sorting and finding minimums and maximums).

Pintos is written in a consistent style. Make your additions and modifications in existing Pintos source files blend in, not stick out.

- **In new source files, adopt the existing Pintos style by preference, but make your code self-consistent at the very least.**
- **Use horizontal and vertical white space to make code readable.** Add a brief comment on every structure, structure member, global or static variable, typedef, enumeration, and function definition.
- **Update existing comments as you modify code.** Don't comment out or use the preprocessor to ignore blocks of code (instead, remove it entirely).
- **Use assertions to document key invariants.**
- **Decompose code into functions for clarity.**
- Code that is difficult to understand because it violates these or other "common sense" software engineering practices will be penalized.

In the end, remember your audience. Code is written primarily to be read by humans. It has to be acceptable to the compiler too, but the compiler doesn't care about how it looks or how well it is written.



Here is the summary of our score criteria for code quality:

Rubric	Deduct points
important discrepancies between your design doc and the actual code	10
poorly designed data structure and code modularization	5
missed comments (you can only omit the comment if the code is self-explained)	2 each, up to 10
Lines exceed 78 characters.	2 each, up to 10
use comments to ignore blocks of code	5
repeated blocks of code	5
compile warning due to the code you implement	2 each, up to 10
code specific to a test case	5
writing large amounts of repetitive code for implementing system calls	2 each, up to 10

[Previous](#)
[Debugging](#)

[Next](#)
[Lab0: Getting Real](#)

Last updated 1 year ago

